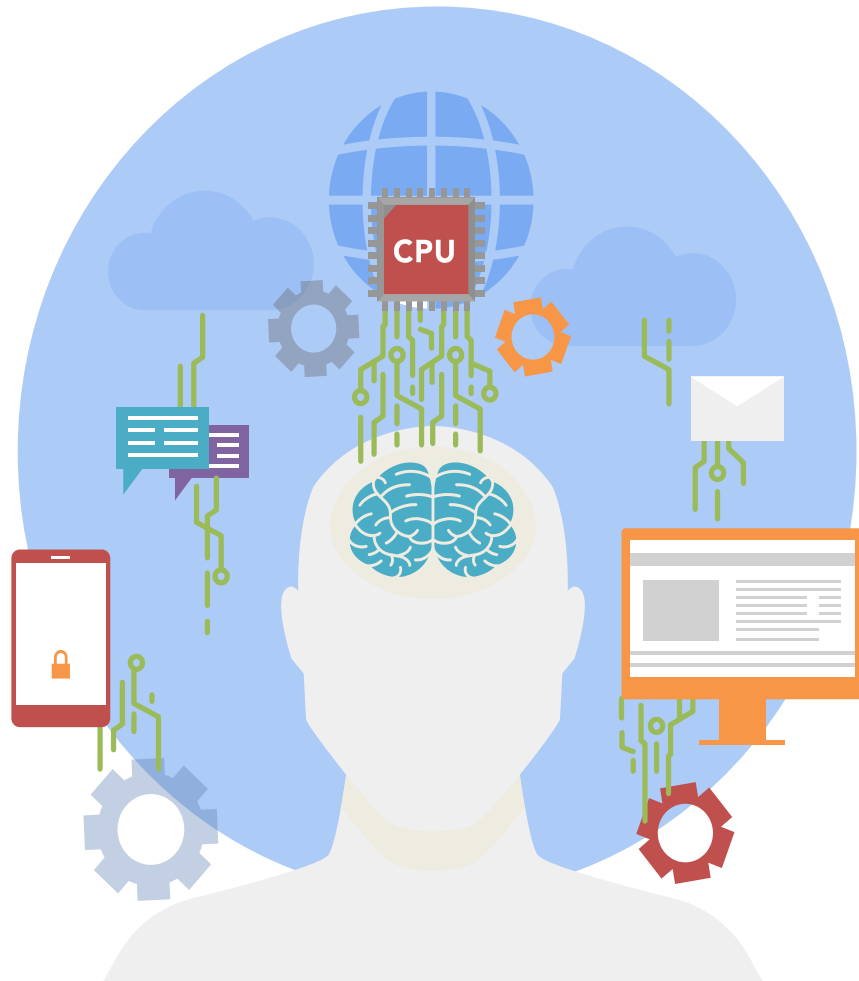
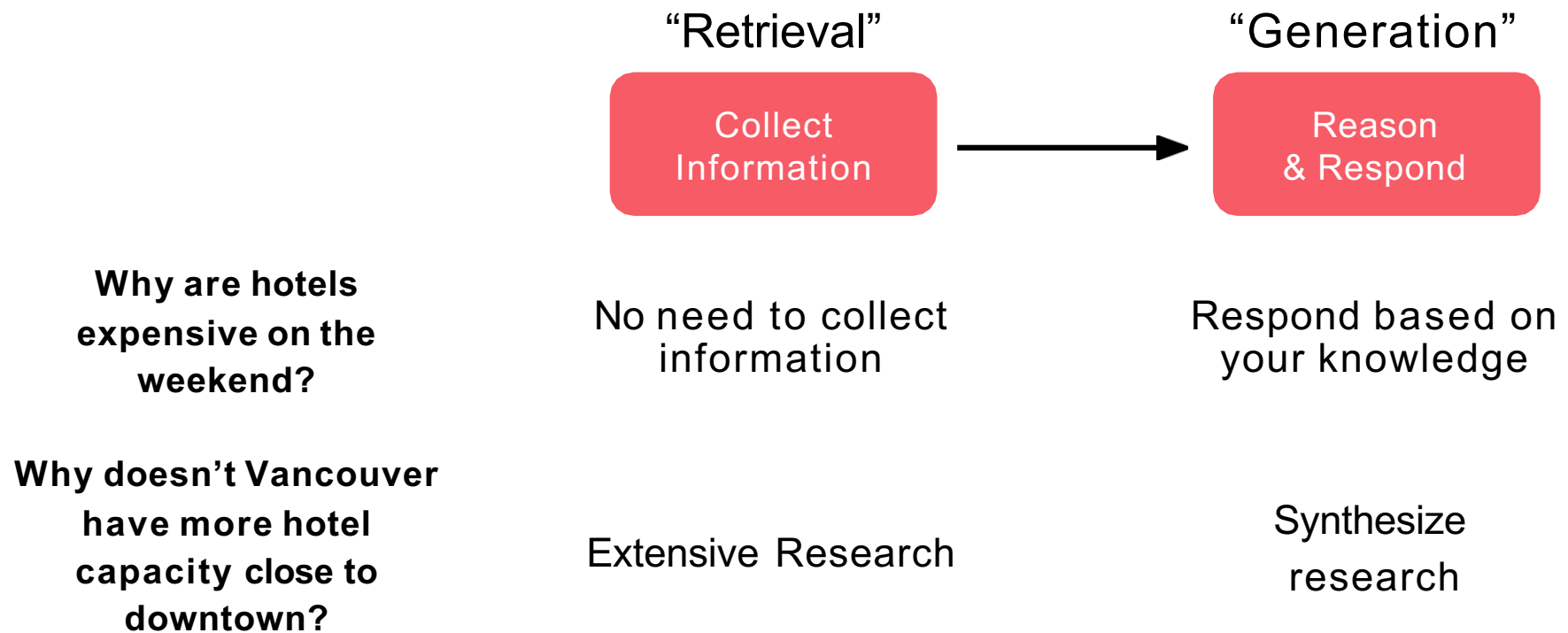




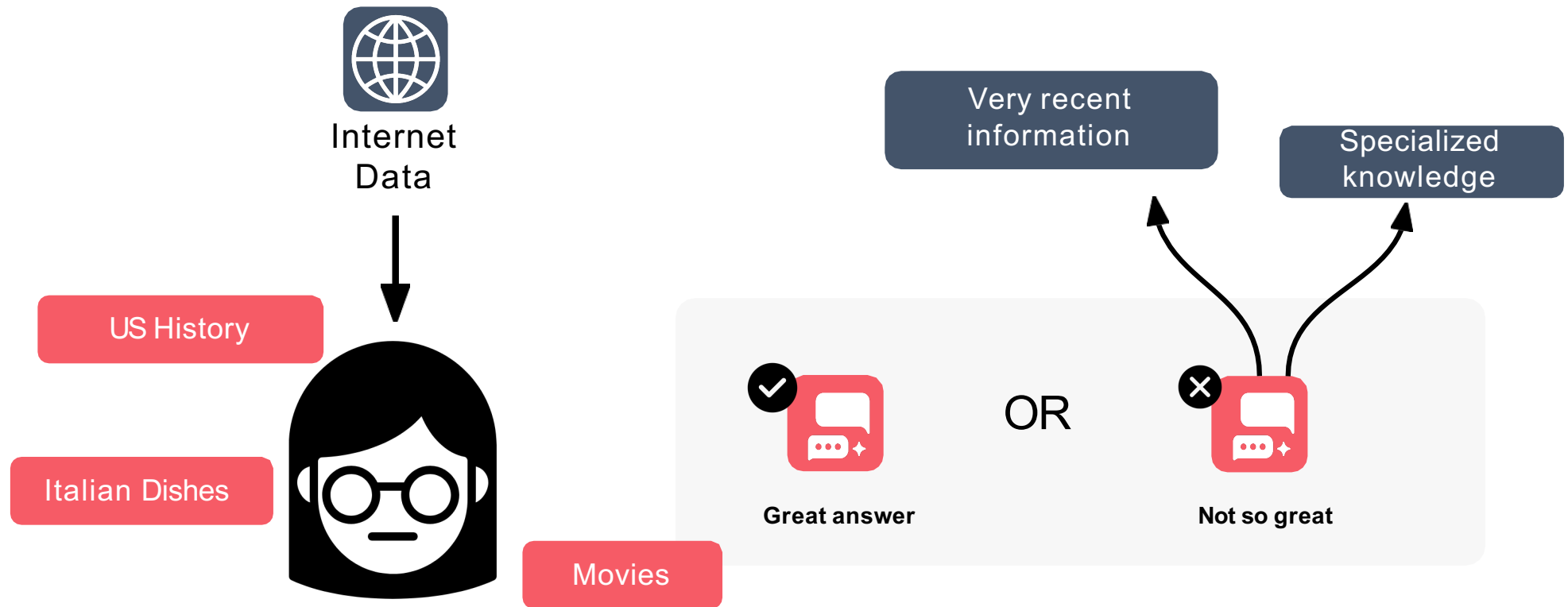
Retrieval Augmented Generation (RAG)

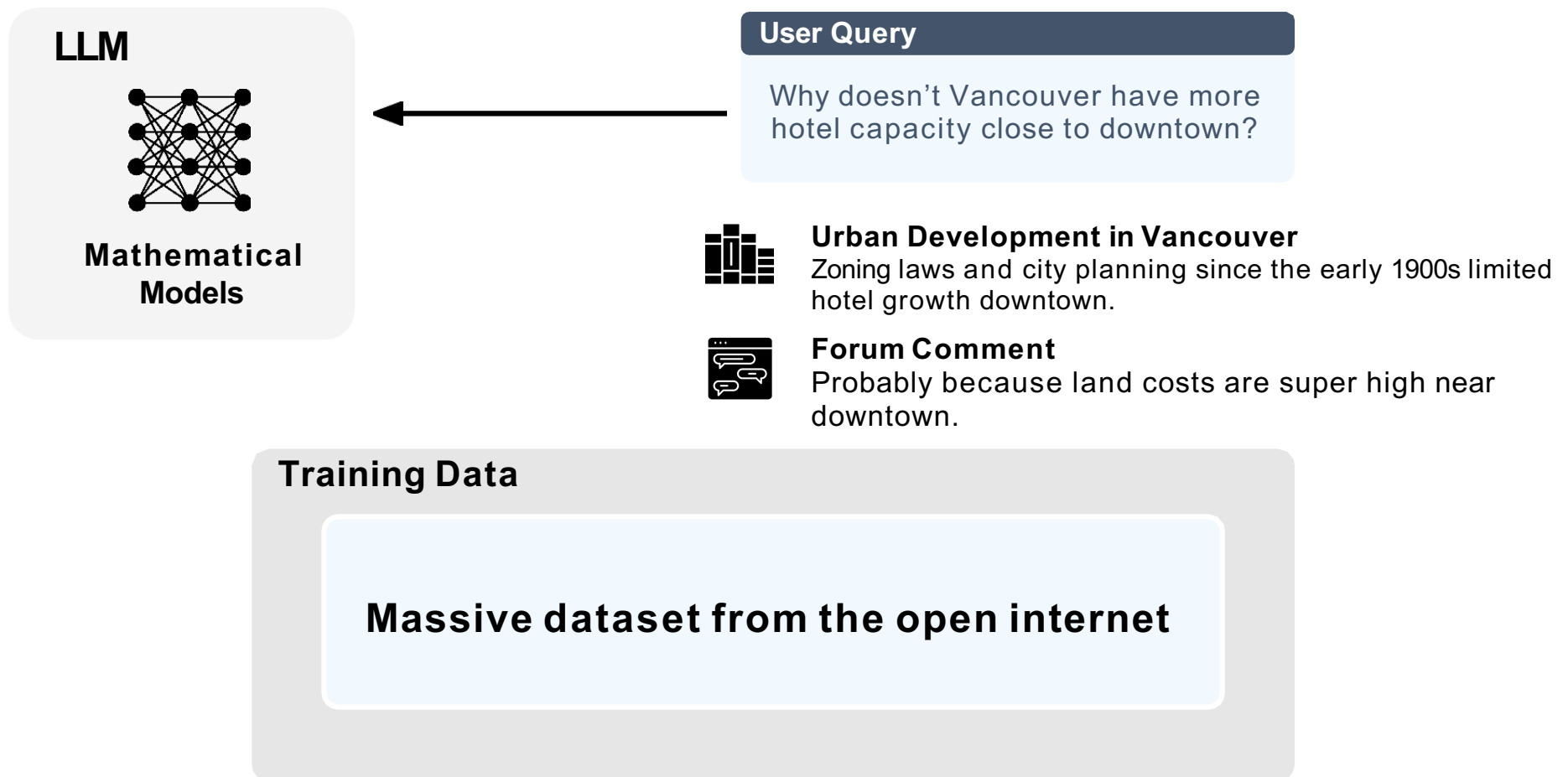
Dr. Tamer Arafa

Two Steps for Answering Questions



Traditional Language Model Usage





What LLMs don't know

Private Databases

LLMs can't access confidential information

Hard to access information

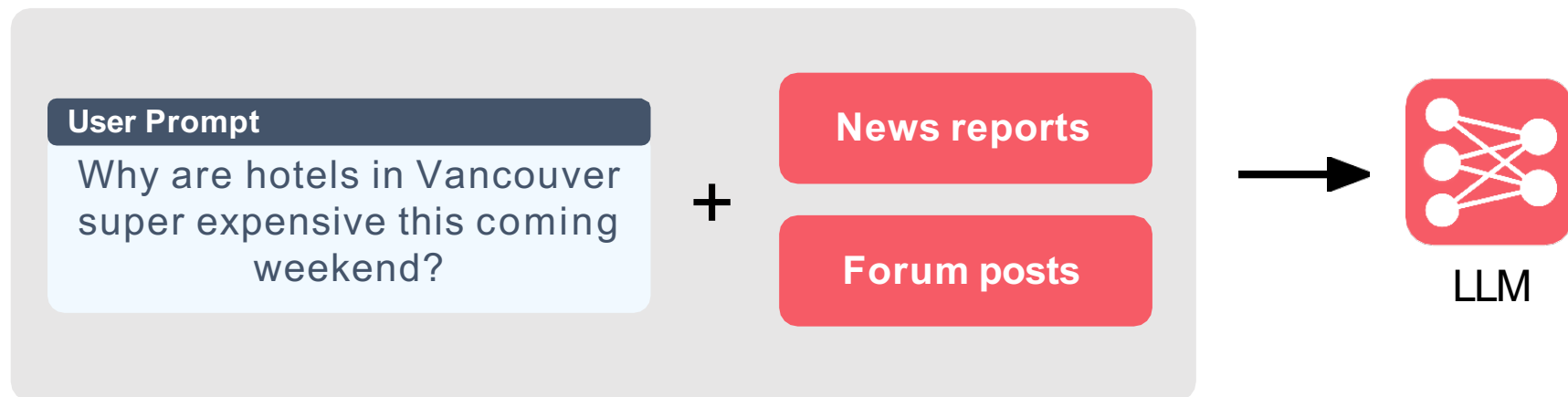
Some information isn't widely available online, making it inaccessible to LLMs.

Real time data

LLMs are trained on past data and don't update automatically

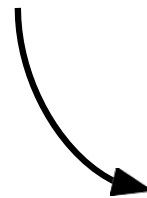
How do you make sure the LLM knows this useful information?

Just put it in the **prompt!**



User Prompt

Why are hotels in Vancouver super expensive this weekend?



RAG System



Augmented Prompt



LLM



Response

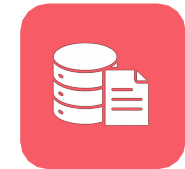
Taylor Swift is performing her Eras Tour in Vancouver. This weekend at BC Place Stadium on December 6–8, 2024

Retriever

- Manages knowledge base of trusted information
- Finds the most relevant Information and shares information with the LLM
- Improves generation

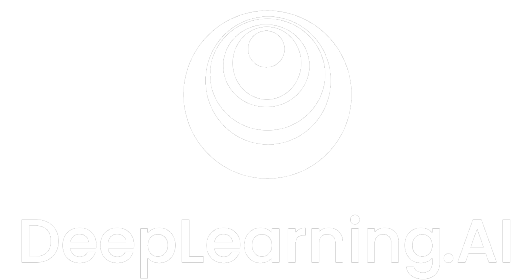


Retriever



Knowledge
Base

Retrieval Augmented Generation



RAG Architecture Overview

Normal LLM Use



Prompt

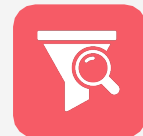
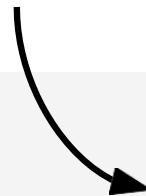


LLM



Response

RAG System



Retriever



Relevant Documents



Knowledge Base

Normal
LLM Use



Prompt

RAG
System



Retriever



Knowledge
Base

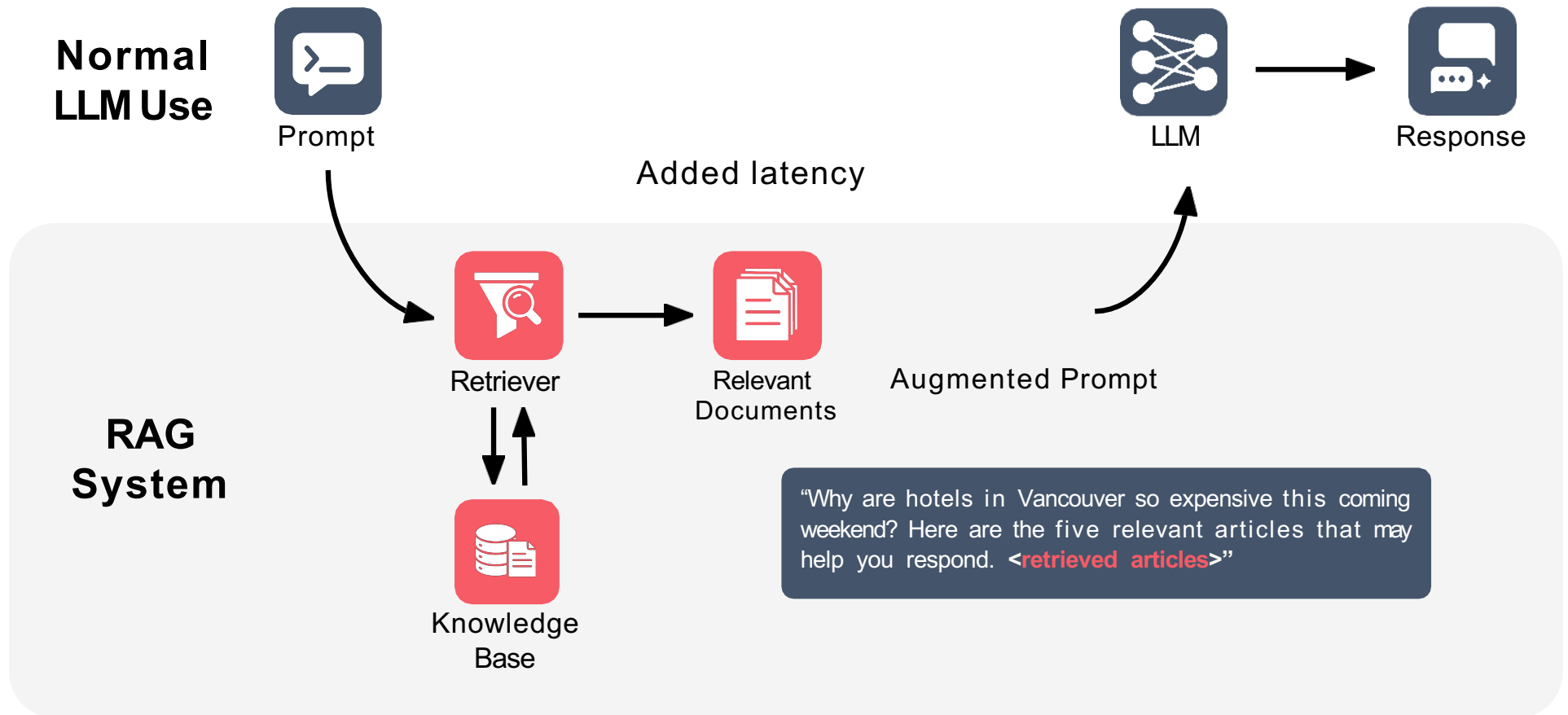


Relevant
Documents



Augmented Prompt

“Why are hotels in Vancouver so expensive this coming weekend? Here are the five relevant articles that may help you respond. <retrieved articles>”



Advantages of RAG

Injects missing knowledge

Adds info not in the training data (e.g. policies, updates)

Reduces hallucinations

Grounds answers with relevant context

Keeps models up to date

Reflects new info by updating the knowledge base

Enables source citation

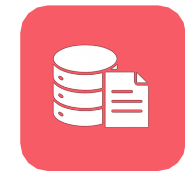
Includes sources for verifiable answers

Focuses model on generation

Retriever finds facts, LLM writes responses



Retriever



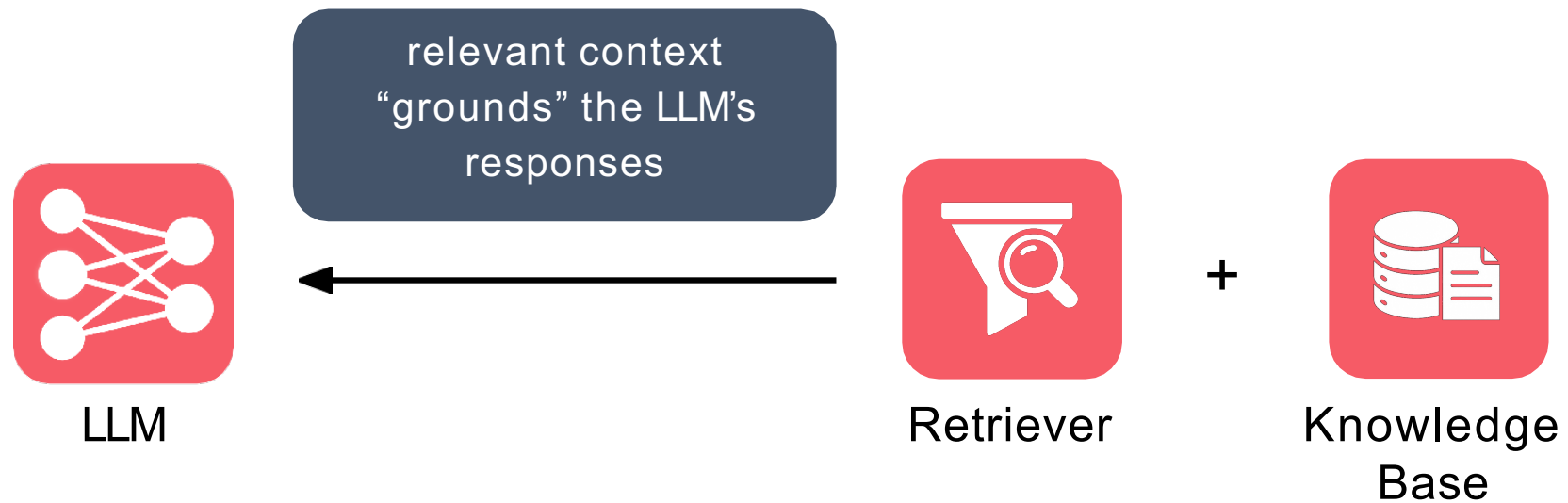
Knowledge
Base

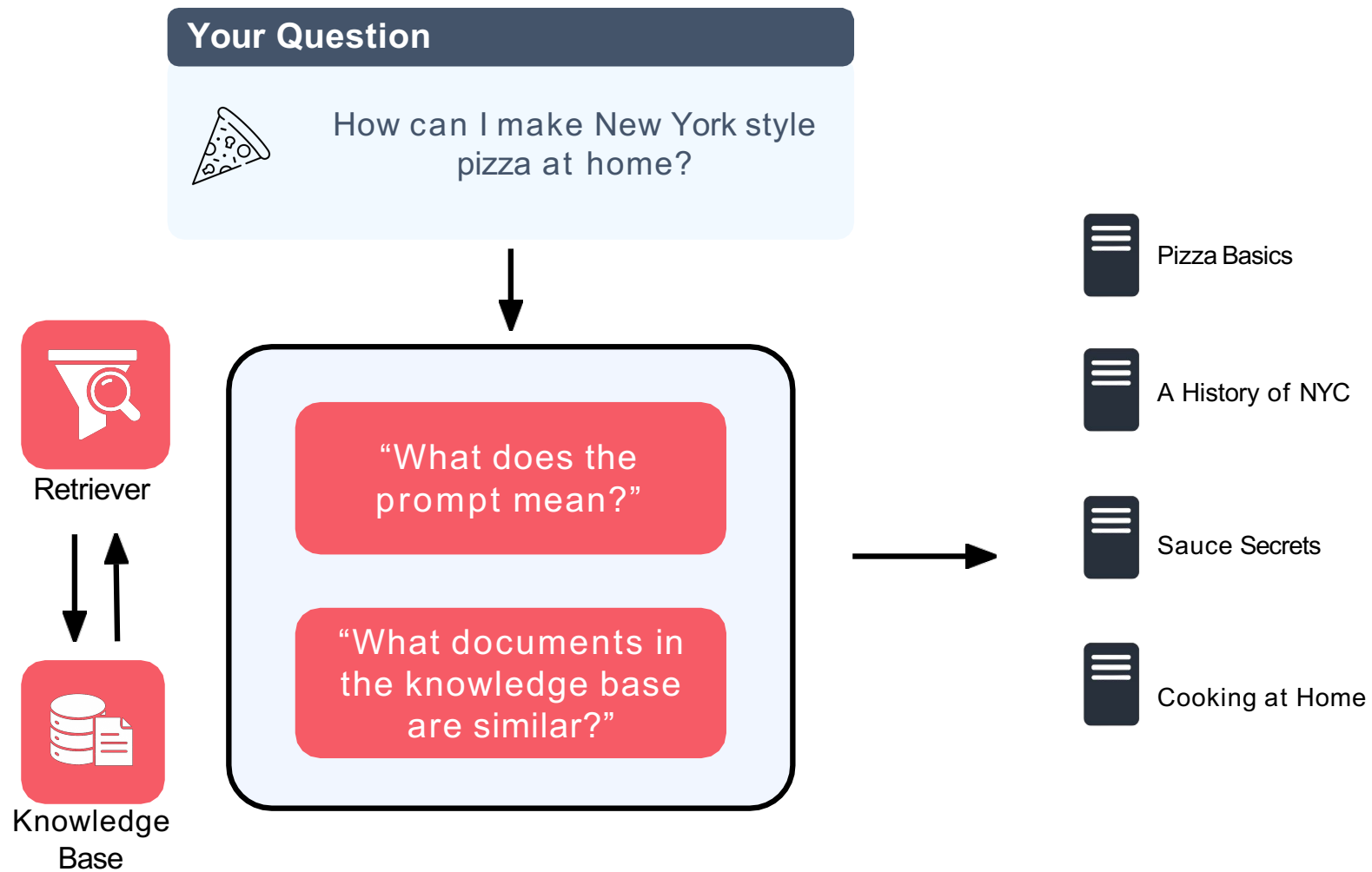
“Why are hotels in Vancouver so expensive this coming weekend? Here are the five relevant articles that may help you respond. <retrieved articles>”

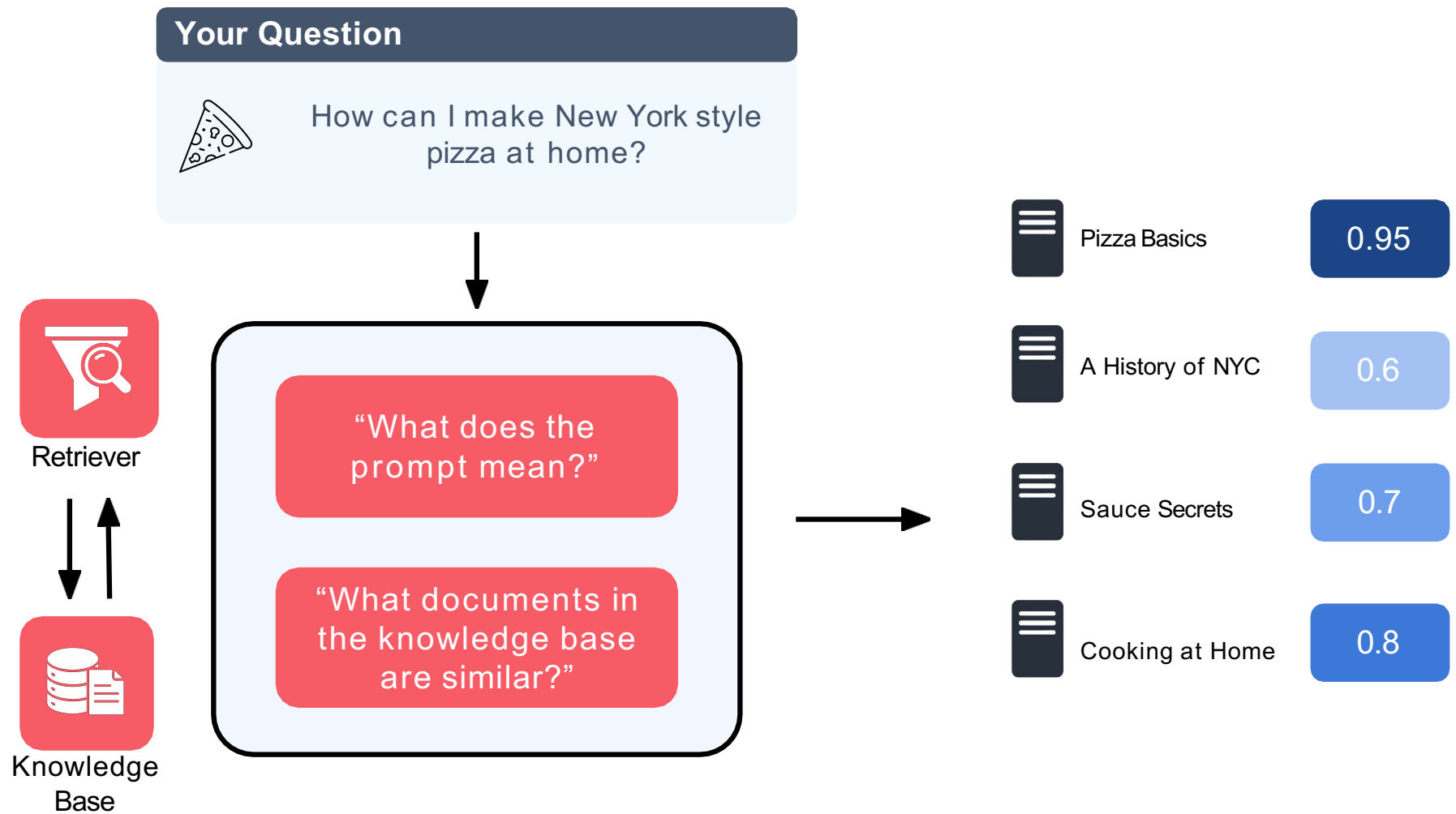
Why LLMs Hallucinate

- **LLMs generate probable word sequences**
LLMs just reproduce statistical patterns from their training data
- **Knowledge gaps cause inaccurate responses**
Responses can “sound right” but aren't true.
- **Truthful ≠ probable**
LLMs are designed to generate "probable" text, not truthful text

How RAG solves the problem

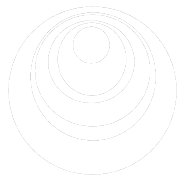






Retriever Tradeoffs

- **Relevance vs irrelevance**
Need to return relevant documents and withhold irrelevant ones
- **Return every document?**
Mountains of irrelevant docs. Wastes context window
- **Return the single highest ranked document?**
Miss valuable information
- **No perfect solution**
Retriever usually doesn't perfectly rank documents
- **Monitor and experiment**
Change settings to find what works



DeepLearning.AI

Information Retrieval Foundations

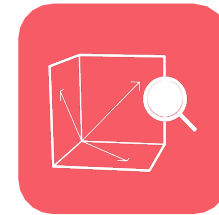


Two Search Approaches



Keyword Search

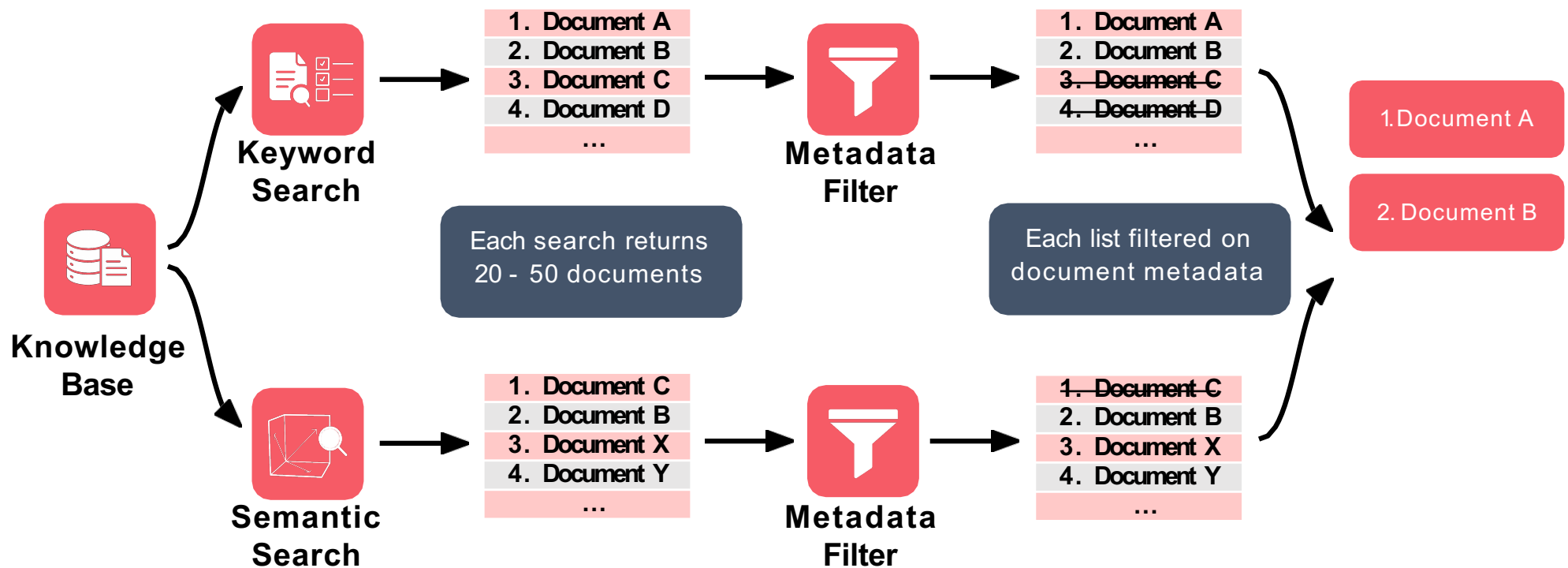
Looks for documents containing the **exact words** found in the prompt.



Semantic Search

Looks for documents with **similar meaning** to the prompt.

Search Techniques



Hybrid Search



Keyword Search

Ensures sensitivity to exact words the user included in the prompt



Semantic Search

Finds documents with similar meaning, even without matching words



Metadata Filtering

Excludes documents based on rigid criteria



High-performing retrievers
balance all three techniques
based on project needs.

Metadata Filtering

Uses rigid criteria to narrow down documents based on metadata like title, author, creation date, access privileges, and more.



The Daily Maple



Title



Publication
Date



Author



Section



Tags

```
SELECT * FROM articles  
WHERE publication_date = '2023-10-01';
```

Return all articles published on specific date

```
section = "Opinion"  
author = "Michael Chen"  
date = June to July 2024
```

Advantages and Limitations

Pros

- Simple to understand and debug
- Fast, optimized, mature, and reliable
- Enforces strict retrieval rules, matching exact filter criteria

Cons

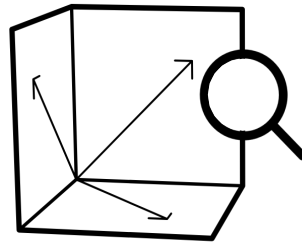
- Not true search
- Rigid, ignores content, and provides no way for ranking
- Useless alone



Keyword search - TF-IDF

Term Frequency – Inverse Document Frequency

Introduction to Keyword Search



Keyword Search



Decades of proven simplicity

The simplicity and effectiveness that has powered retrieval for decades makes keyword search a key component of modern RAG systems.



How can I make New York style pizza at home?



**Keyword
Search**



Doc 1

Use bread flour for New York pizza dough.



Doc 2

New York pizzerias stay open late for home delivery.

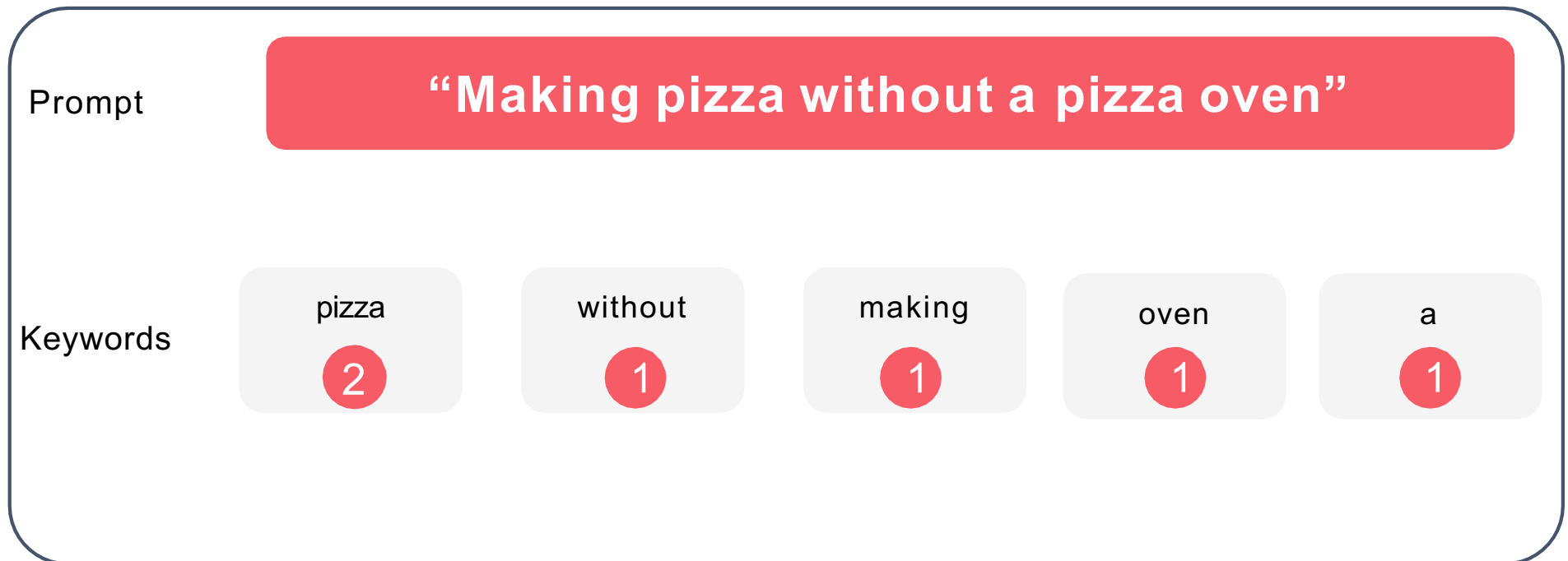


Doc 3

At New York Pizza Mexico, they serve pizza with jalapeño

Bag of Words

Word order is ignored, only word presence and frequency matter



Bag of words

Sparse Vectors

“Making pizza without a pizza oven”

making
1

pizza
1

cake
0

pan
0

without
1

drink
0

pasta
0

a
1

oven
1

pan
0

pasta
0

burger
0

taco
0

with
0

salad
0

blend
0

tea
0

the
0

Most words aren't used. The bag of words is sparse, with few non-zero entries.

“Making pizza without a pizza oven”

	5/5 Doc 1	3/5 Doc 2	1/5 Doc 3	4/5 Doc 4	4/5 Doc 5
making	+1			+1	+1
pizza	+1		+1		
without	+1	+1		+1	+1
a	+1	+1		+1	+1
oven	+1	+1		+1	+1

Frequency Based Scoring

Example Query: pizza oven

Document 1

Homemade **pizza** in **oven** is better
than frozen **pizza**

Contains: Pizza (2x) Oven (1x)

Simple Scoring = 2 points

TF Scoring = 3 points

Document 2

Wood-fired **oven** is a better **oven** than
a stone **oven** for cooking **pizza**

Contains: Pizza (1x) Oven (3x)

Simple Scoring = 2 points

TF Scoring = 4 point

Normalized TF Scoring

Longer documents may contain **keywords many times** simply because they are **longer**.

Solution: Normalize by document length

Score = (Number of keyword occurrences) / (Total words in document)

TF-IDF

Basic TF scoring treats **all words equally**, whether they're common filler words or rare, meaningful terms.

Solution: Weight terms using “inverse document frequency” (IDF).

Score = $\text{TF}(\text{word}, \text{doc}) \times \log(\text{Total docs} / \text{Docs containing word})$

1 Count Documents

For each word

making

pizza



docs word appears in

total docs

1

Count Documents

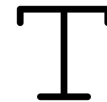


Pizza

Appears in 5 out of 100 documents

DF = 5/100

0.05



The

Appears in all 100 documents

DF = 100/100

1.0

1

Count Documents

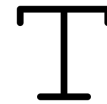


Appears in 5 out of 100 documents

Pizza

$DF = 5/100$

0.05



Appears in all 100 documents

The

$DF = 100/100$

1.0

2

Flip to reward rare words

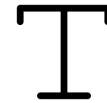


High Score

Pizza

$IDF = 1/0.05$

20



Low Score

The

$DF = 1/1.0$

1.0

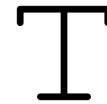
1

Count Documents

Appears in 5 out of 100 documents

Pizza

$DF = 5/100$

0.05**The**

Appears in all 100 documents

$DF = 100/100$

1.0

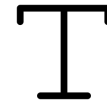
2

Flip to reward rare words

High Score

Pizza

$IDF = 1/0.05$

20**The**

Low Score

$DF = 1/1.0$

1.0

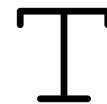
3

Apply log

Still higher

Pizza

$\log(1/20)$

1.30**The**

Too common, no weight

$\log(1)$

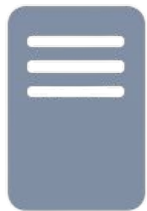
0

TF-IDF

**TF (Term Frequency) = (Number of keyword occurrences) /
(Total words in document)**

TF-IDF = TF × $\log_{10}$ (Total docs / Docs containing word)

“Making pizza without a pizza oven”



High Scoring Document

Pizza
oven

1.3

1.1



Low Scoring Document

a
without

0

0.1

Documents with **rare keywords** score **higher** than documents with common words

Keyword Search Overview



Match documents by **keyword frequency**

Sparse
Vectors



Scored



Ranked

TF-IDF

- Keyword rarity
- Term frequency
- Document length

BM25

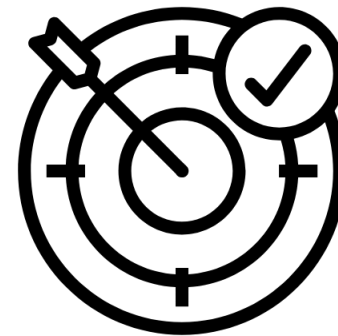
Most commonly used

- Document length normalization
- Term Frequency Saturation

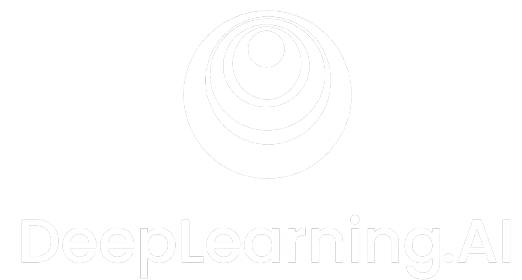
Keyword Search Strengths



Simplicity



**Guaranteed keyword
matching**



Semantic search introduction

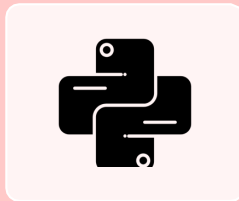
In order to improve on lexical search it's necessary to capture not only the presence of words, but their **meaning**.

Cannot match synonyms

“happy”

“glad”

Incorrectly matches different meanings



Semantic Search vs. Keyword Search

- Prompt and documents each get a vector
- Vectors compared to generate scores
- The main difference is how vectors are assigned
 - **Keyword Search:** count words
 - **Semantic Search:** use embedding model

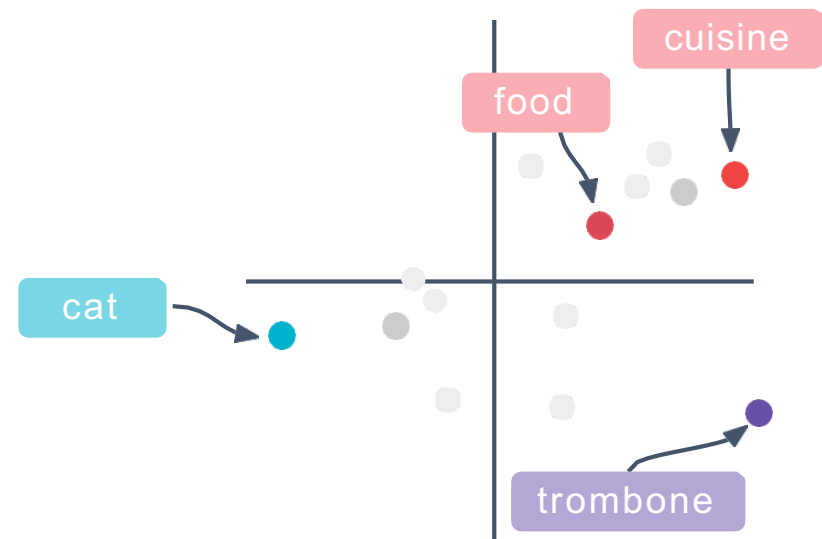
Understanding Embedding Models

Embedding models map tokens, to a location in space. This location is represented by a **vector**.

“**Pizza**” → vector [3, 1]

“**Bear**” → vector [5, 2]

In two dimensions, these can be represented as points



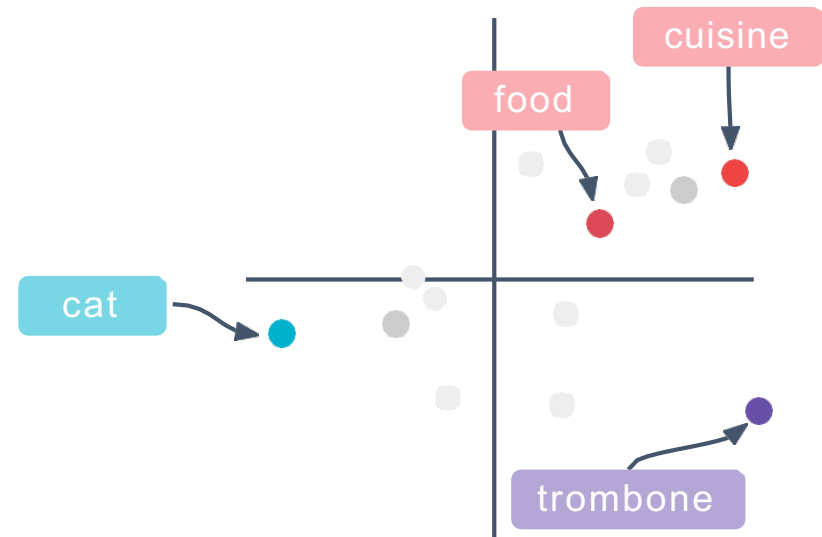
Vector Space

Understanding Embedding Models

No simple interpretation of X
and Y axis

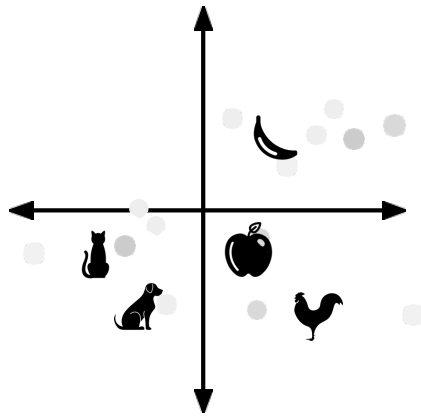
...instead...

points “float around” in
space and similar words
cluster together

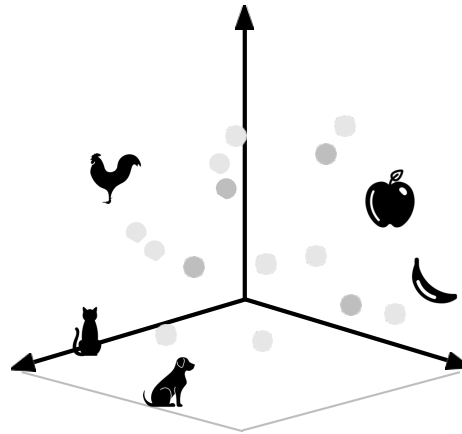


Vector Space

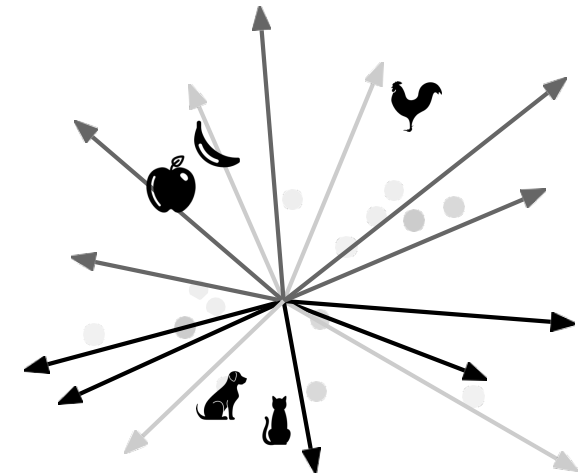
Understanding Embedding Models



2 dimensions



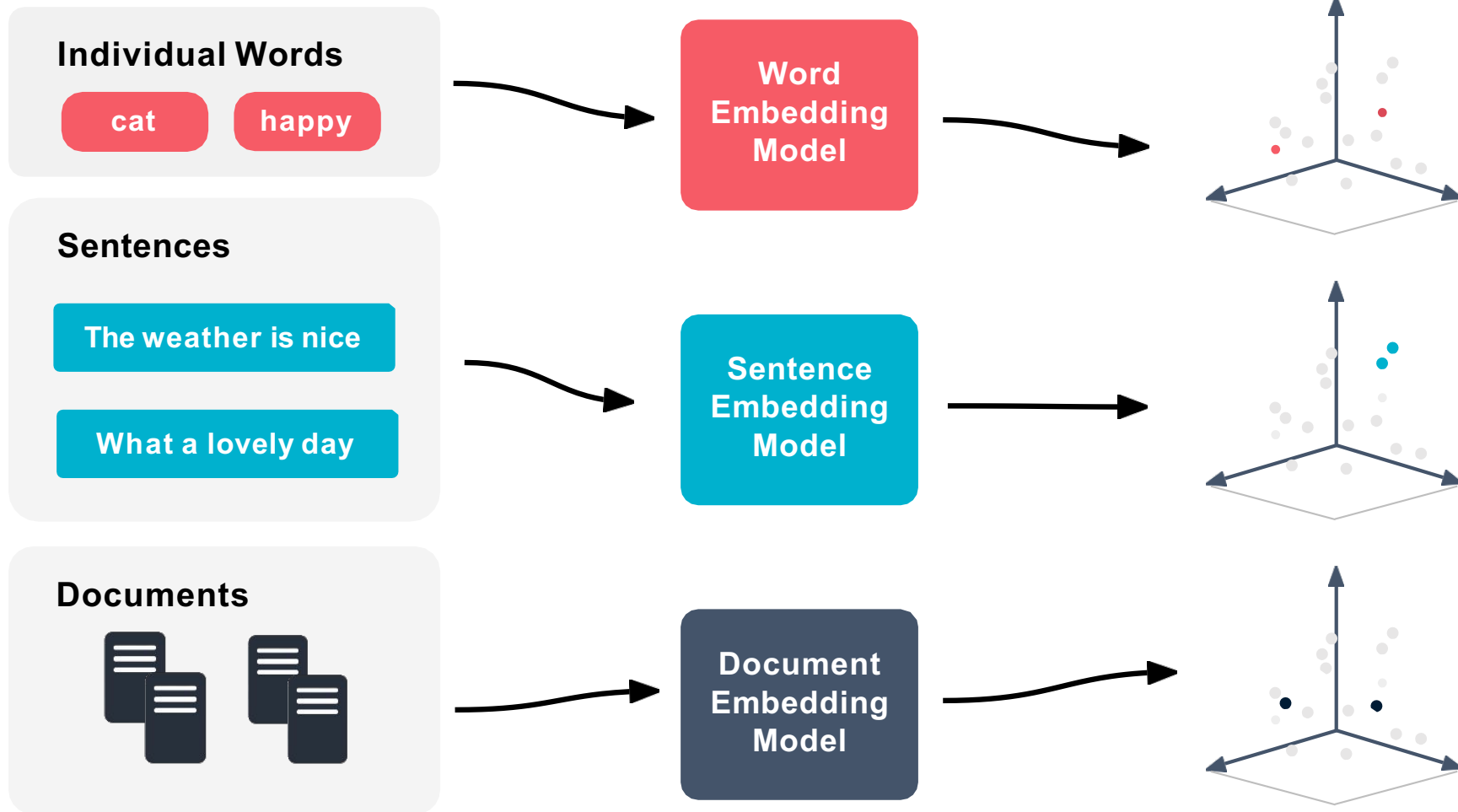
3 dimensions



100 - 1,000+ dimensions

More dimensions means more room
to form clusters and capture
nuanced relationships

Same principles hold
Close vectors, similar meanings

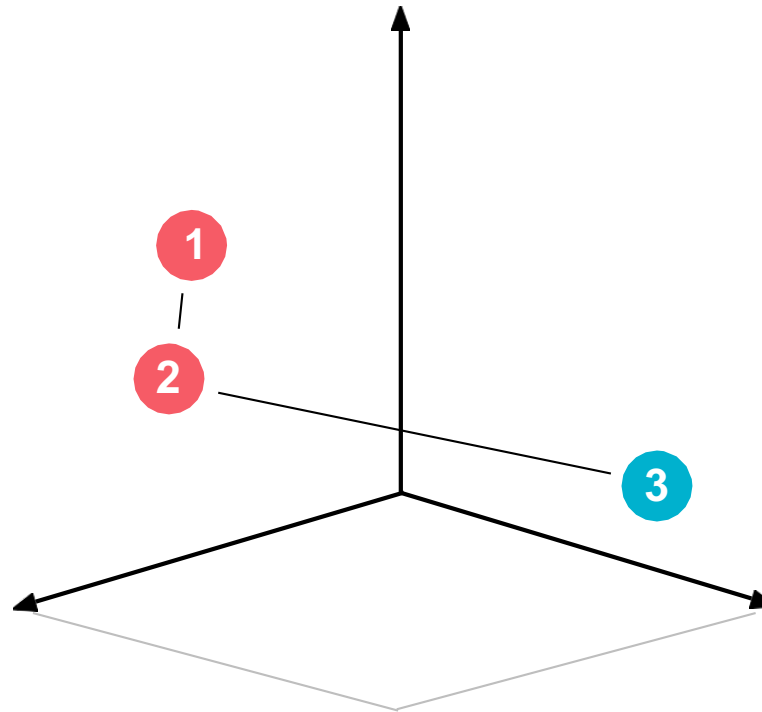


Sentence Embedding Example

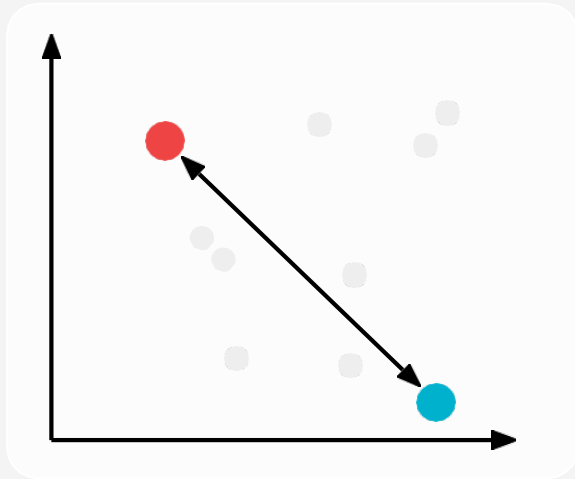
1 He spoke softly in class

2 He whispered quietly
during class

3 Her daughter brightened
the gloomy day



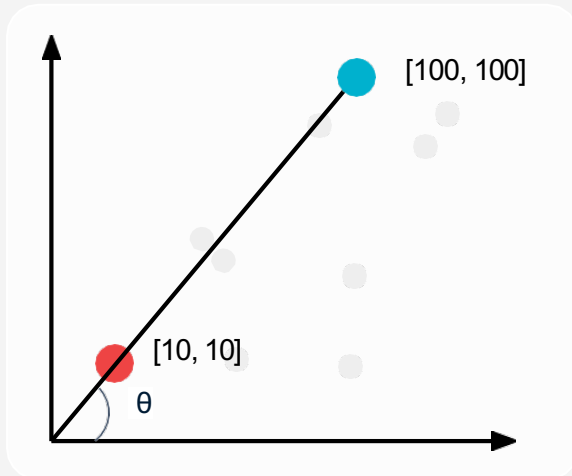
Measuring Vector Distance



Euclidean Distance

Measures how far apart two vectors are by drawing a straight line from one vector to the other - the shortest possible distance between them.

Measuring Vector Distance



Cosine Similarity

Looks at the similarity in the **direction** of two vectors, regardless of whether they're close to one another in space

-1

Opposite
direction

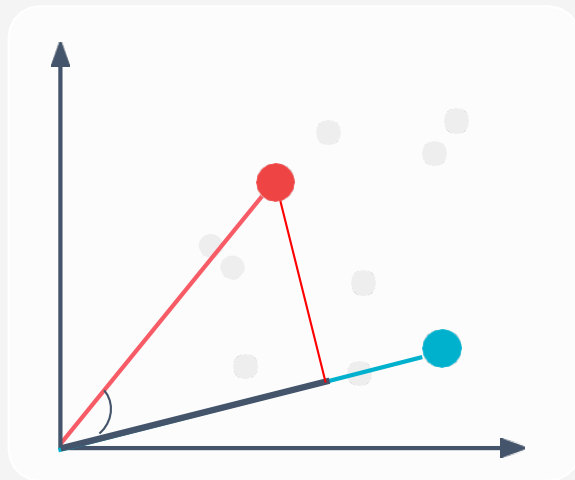
0

Perpendicular

1

Same
direction

Measuring Vector Distance



Dot product

Measures the length of the projection of one vector onto another.

Negative

Opposite
direction

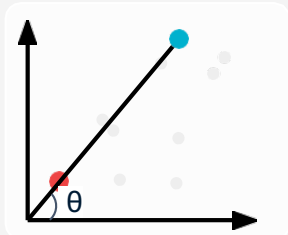
0

Perpendicular

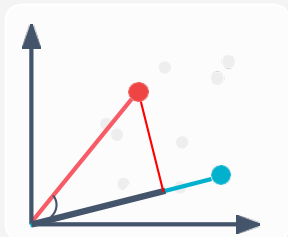
Positive

Same
direction

Measuring Vector Distance



Cosine Similarity



Dot product

-1

Opposite
direction

0

Perpendicular

1

Same
direction

Higher values, closer vectors

Negative

Opposite
direction

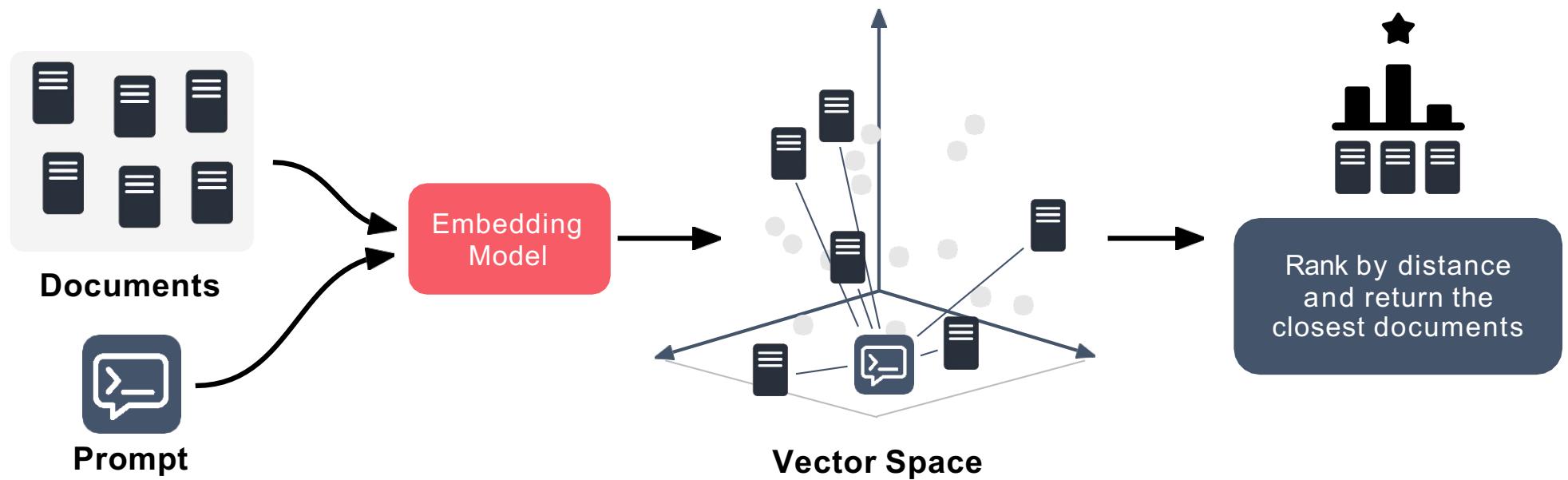
0

Perpendicular

Positive

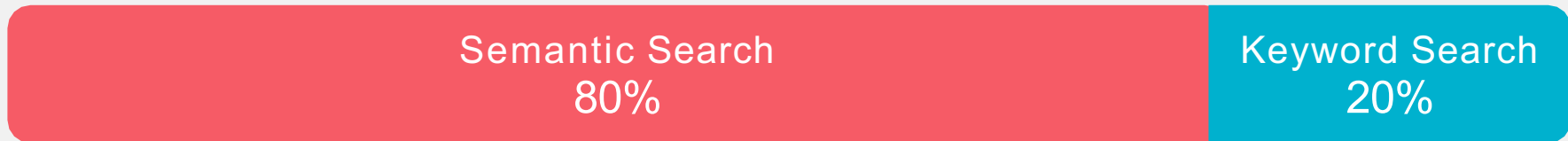
Same
direction

Semantic Search

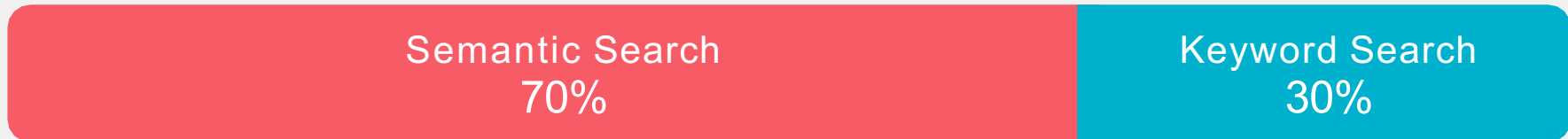


Beta: Weighting Semantic vs. Keyword

$\beta = 0.8$



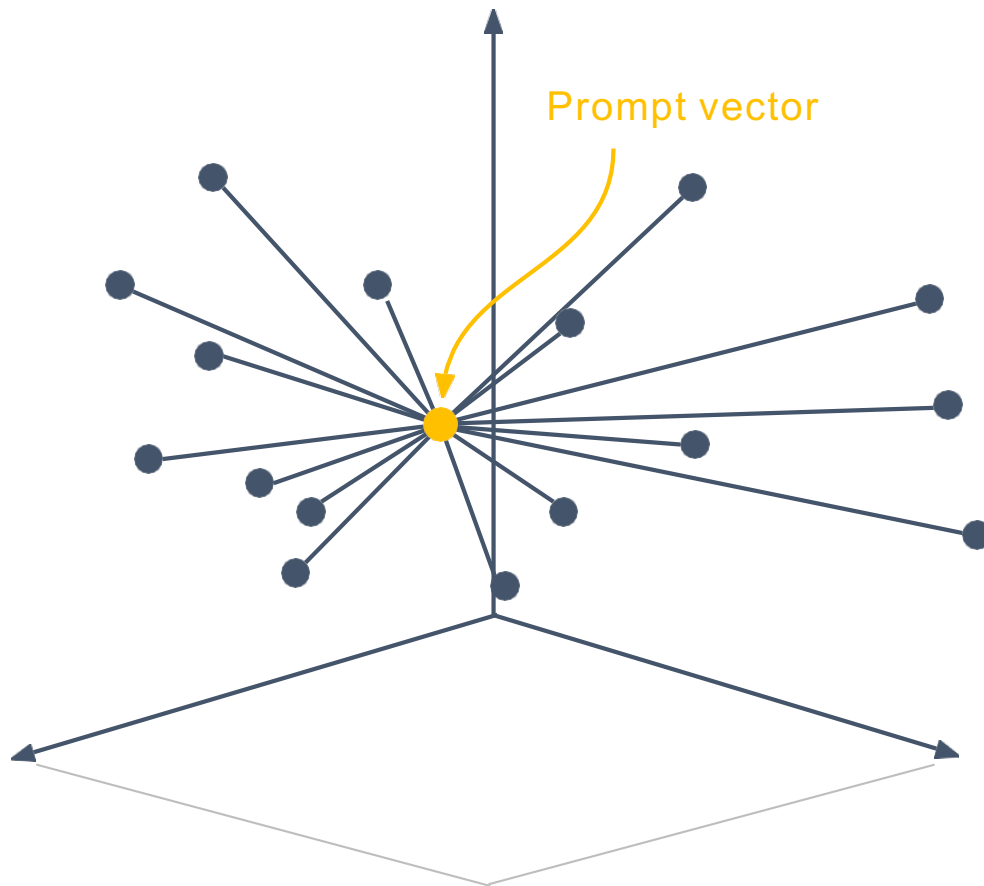
$\beta = 0.7$



If exact keyword matching is important, set a lower beta

Approximate nearest neighbors algorithms (ANN)

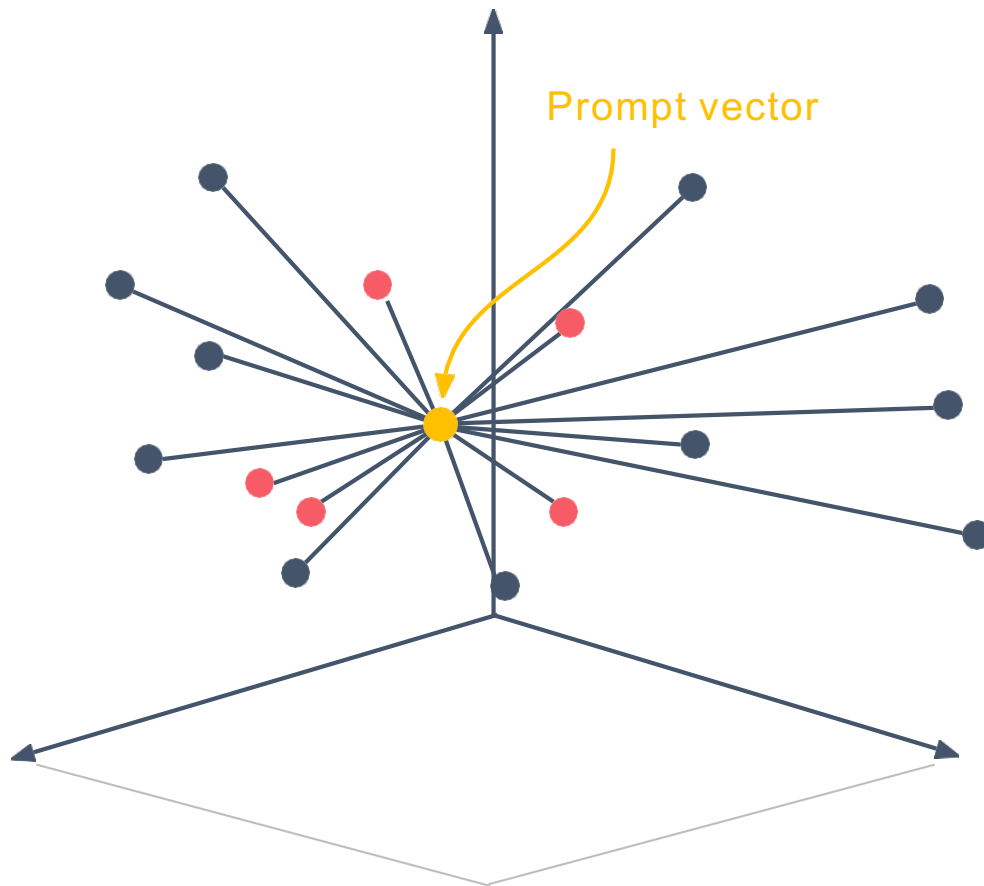
Basic Vector Retrieval - KNN



KNN: K Nearest Neighbors

- **Vectorize** all documents and prompt
- **Compute** distances to all document vectors
- **Sort** by distance

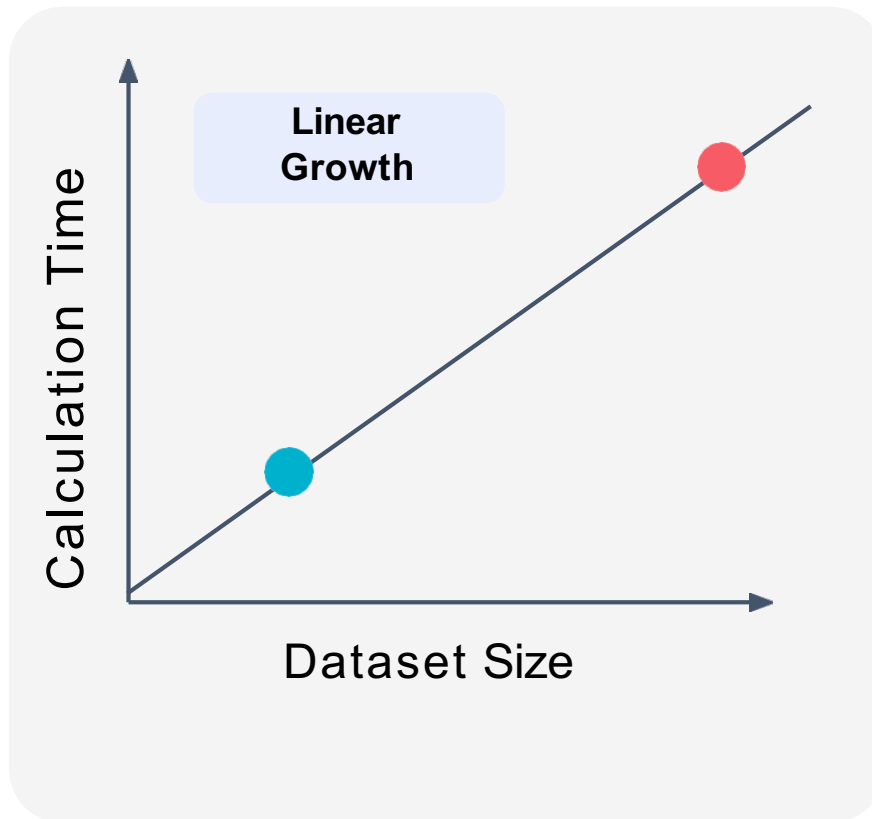
Basic Vector Retrieval - KNN



KNN: K Nearest Neighbors

- **Vectorize** all documents and prompt
- **Compute** distances to all document vectors
- **Sort** by distance
- **Return** closest elements

Scaling Challenges

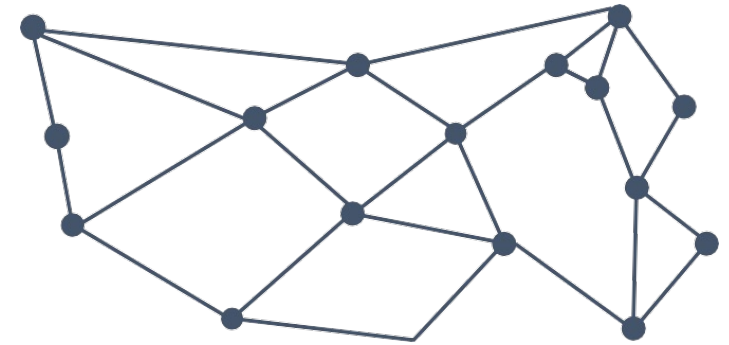


- 1,000 docs = 1,000 distance calculations per search
- 1B docs = 1B distance calculations every time

If you want to build high-performing retrievers, you'll need a better approach!

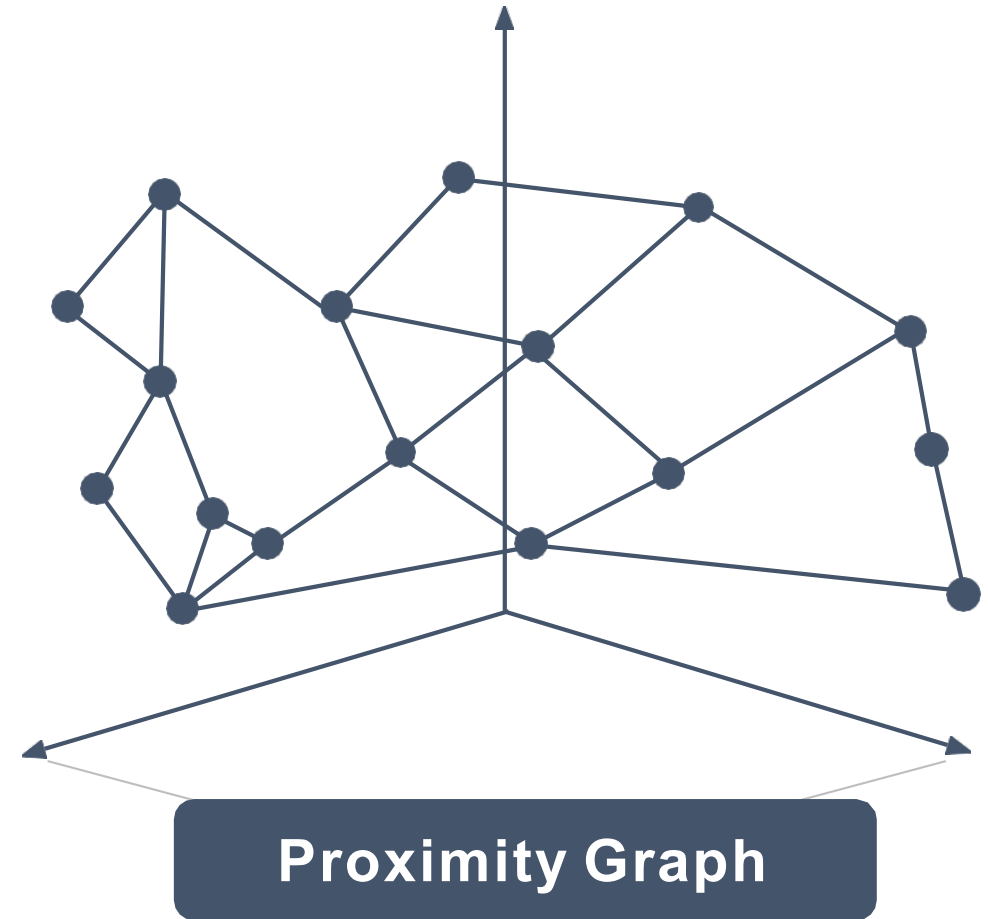
Approximate Nearest Neighbors

- ANN is significantly faster than KNN
- Rely on additional data structures
- Not guaranteed to find the absolute closest documents

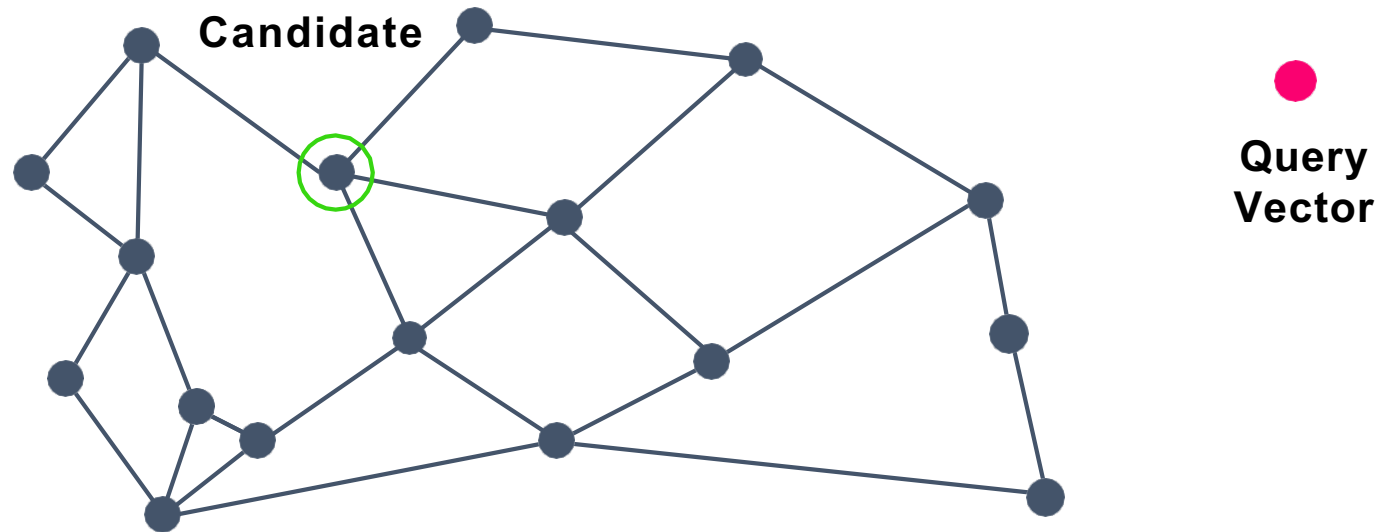


Navigable Small World

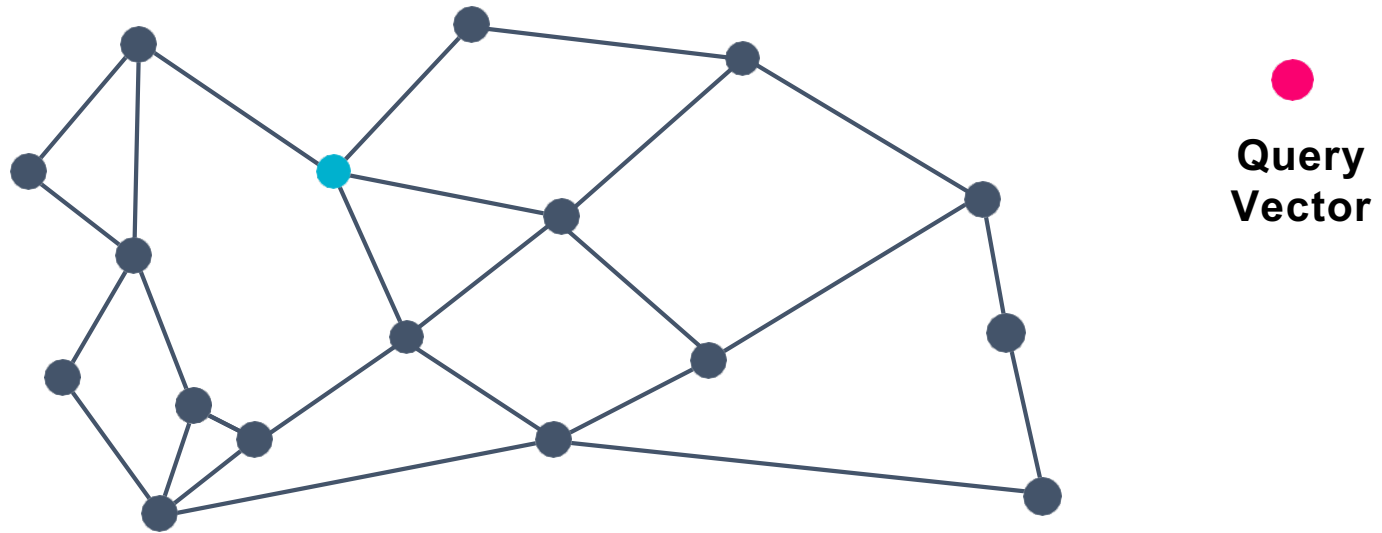
- Compute distances between all document vectors
- Add one node to the graph for each document
- Connect each node to its nearest neighbors
- Can traverse the graph moving along edges between neighboring documents



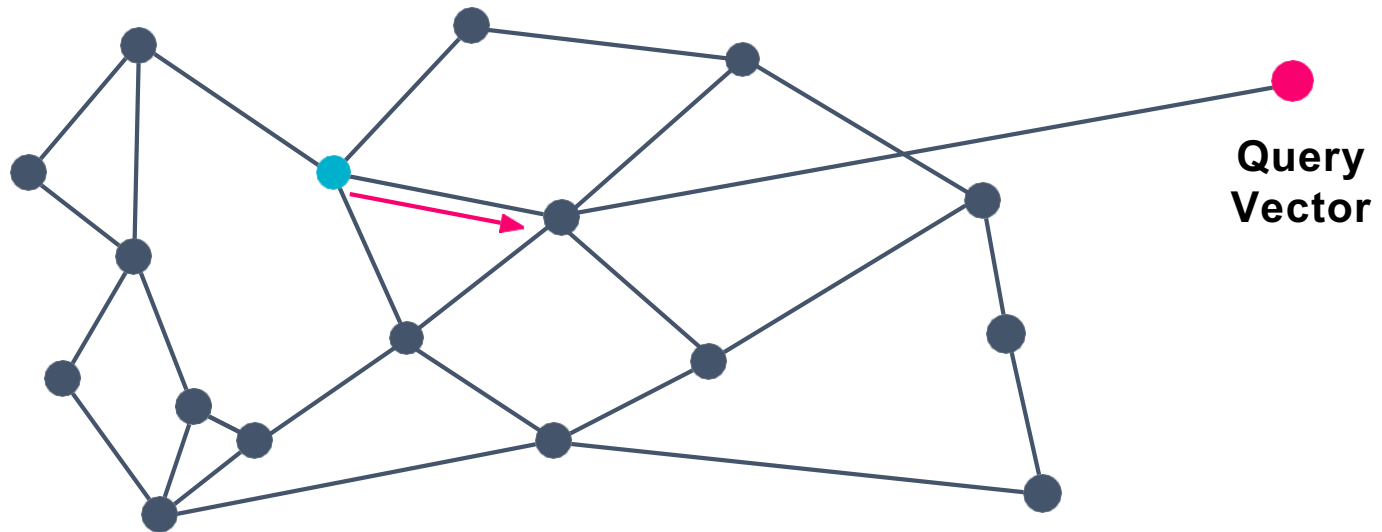
Query Entry Point



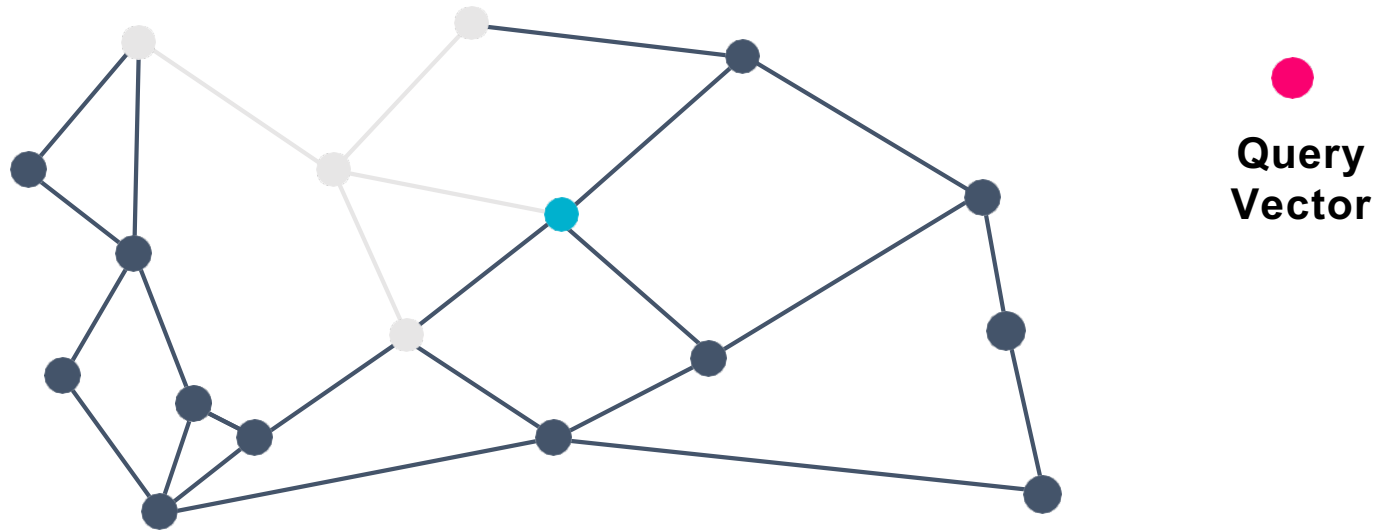
Search Algorithm



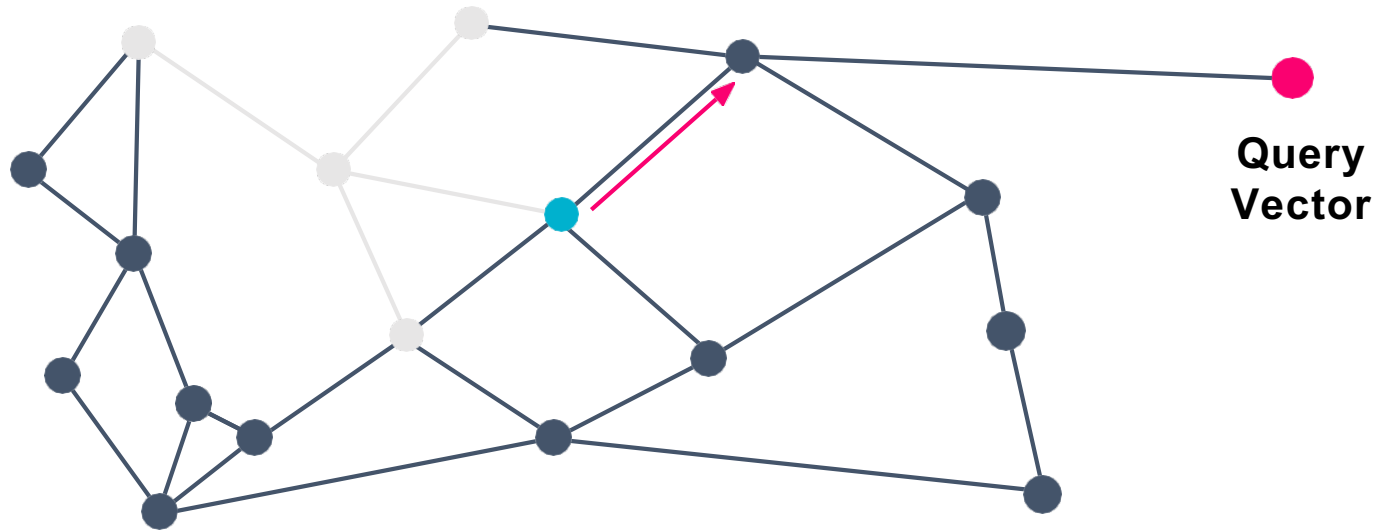
Search Algorithm



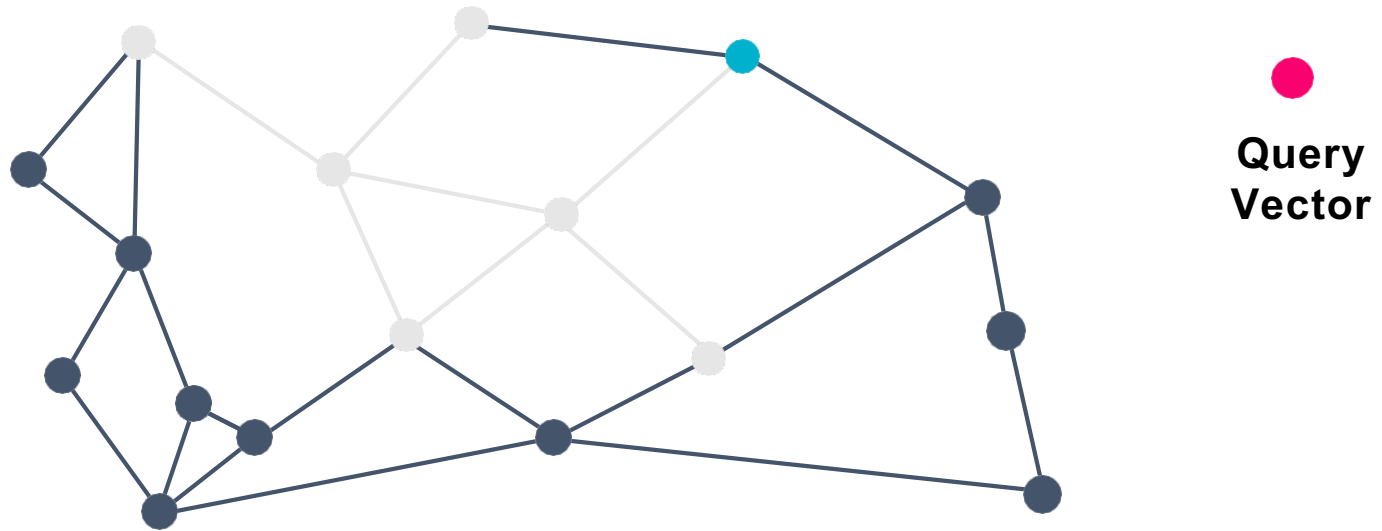
Search Algorithm



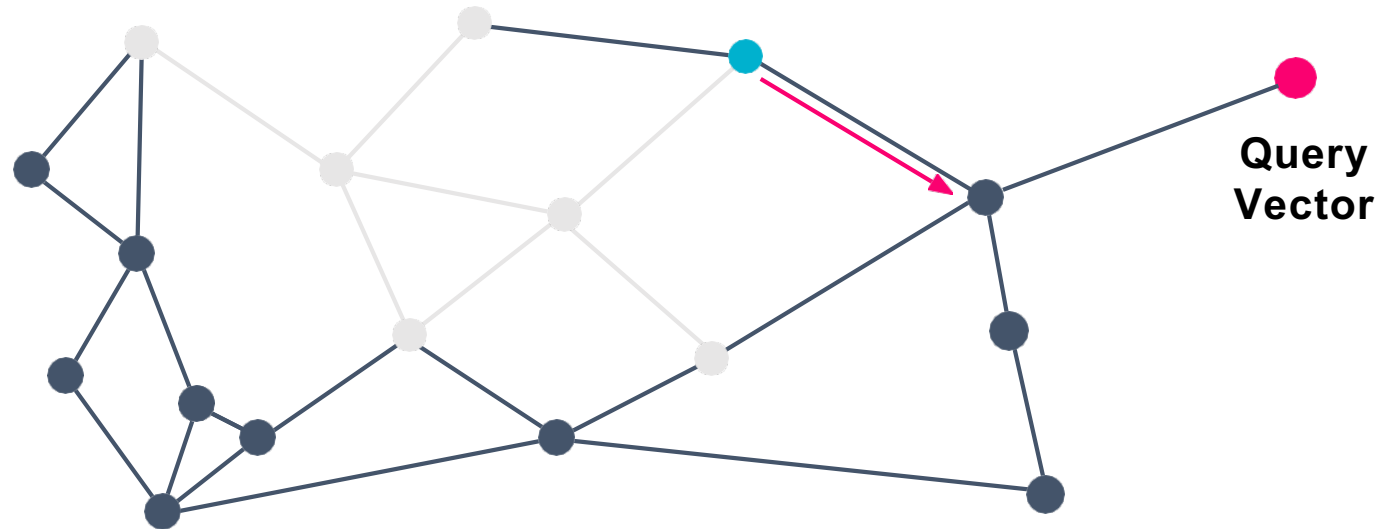
Search Algorithm



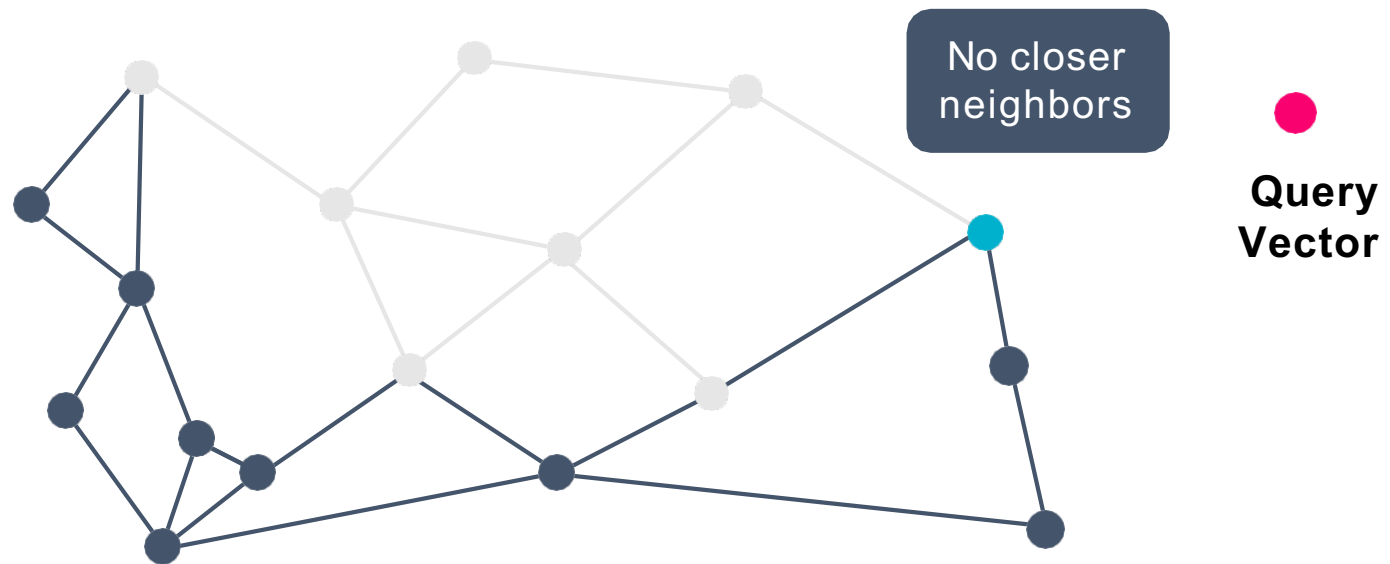
Search Algorithm



Search Algorithm



Search Algorithm



Hierarchical Improvement

- Hierarchical Navigable Small World (HNSW)
enhances Navigable Small World by speeding up
early parts of the search
- Relies on a hierarchical proximity graph

3 Layer 3

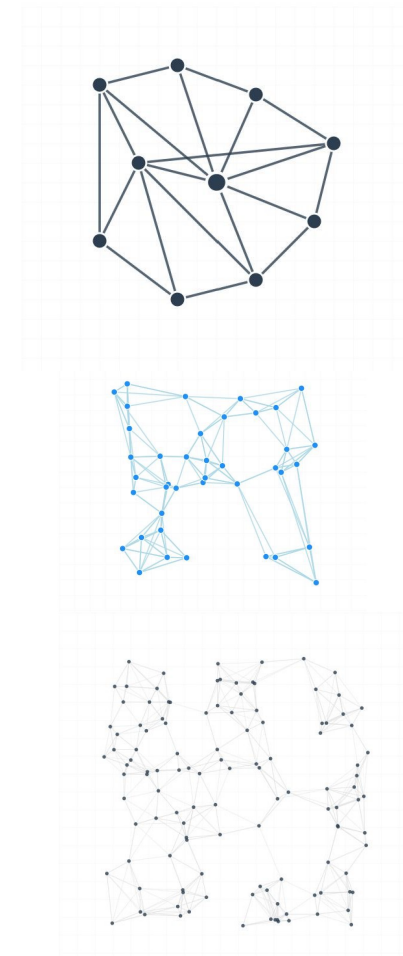
Randomly drop to just **10 vectors** and create a proximity graph for fast navigation at the highest level

2 Layer 2

Randomly drop to **100 vectors** and build a new proximity graph for intermediate navigation

1 Layer 1

Contains all **1,000 vectors** with complete proximity graph for precise final search



3 Layer 3

Choose random candidate vector and search top layer to get as close as you can in this layer



Make big jumps early

2 Layer 2

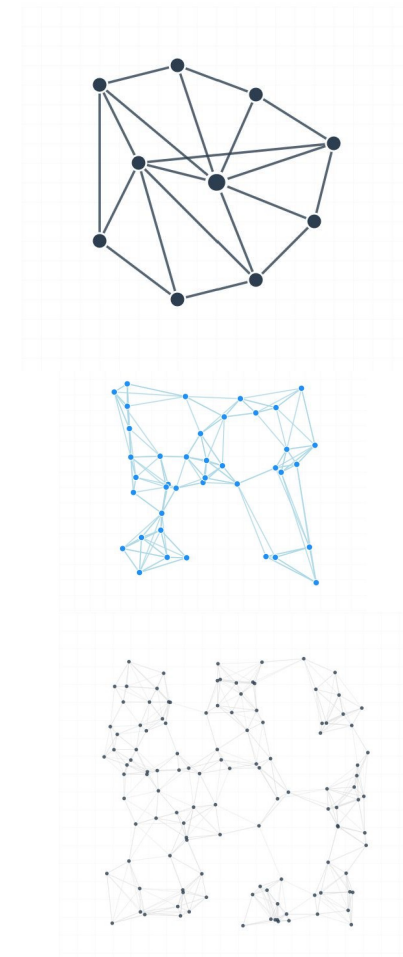
Start at best candidate from layer 3, and complete normal search through the layer 2 proximity graph



Close to prompt vector once in Layer 1

1 Layer 1

Start at best candidate in layer 2, and complete normal search through the layer 1 proximity graph



Vector databases

Vector Database

Designed for Vector Search

Designed to store high-dimensional vectors and perform vector search using ANN algorithms

Outperform Relational Databases

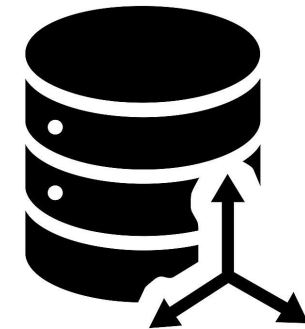
Relational databases perform vector search similarly to an inefficient KNN search

Optimized for ANN Search

Designed to build HNSW indexes and compute vector distances. They scale well and operate quickly

Weaviate

Popular open-source vector database you'll use in this course



**Vector
Database**



Weaviate

Typical Vector Database Operations

- Database setup
- Load documents
- Create sparse vectors for keyword search
- Create dense vectors for semantic search
- Create HNSW index to power ANN algorithm
- Run searches!

Connect to Database and Create a Collection

```
from weaviate.classes.config import Configure, Property, DataType
```

```
client.collections.create(
```

```
    "Article",
```

```
    vectorizer_config=Configure.Vectorizer.text2vec_openai(),
```

```
    properties=[ # properties configuration is optional
```

```
        Property(name="title", data_type=DataType.TEXT),
```

```
        Property(name="body", data_type=DataType.TEXT),
```

```
    ]
```

```
)
```

Creates collection

Specifies vectorizer

Specifies Properties

Adding objects to a collection

```
with collection.batch.fixed_size(batch_size=200) as batch:
    for data_row in data_rows:
        batch.add_object(
            properties=data_row,
        )
        if batch.number_errors > 10:
            print("Batch import stopped due to excessive errors.")
            break
```

Batch adding data

Detecting errors

```
failed_objects = collection.batch.failed_objects
if failed_objects:
    print(f"Number of failed imports: {len(failed_objects)}")
    print(f"First failed object: {failed_objects[0]}")
```

Handling failures

Vector Search in Action

```
from weaviate.classes.query import MetadataQuery
```

```
articles = client.collections.get("Article")
```

```
response = articles.query.near_text(  
    query="hotel capacity in downtown Vancouver",  
    limit=2,  
    return_metadata=MetadataQuery(distance=True)  
)
```

```
for o in response.objects:  
    print(o.properties)  
    print(o.metadata.distance)
```

Specifies Collection

Performs vector
search with
"near text"

Asks for vector distances

Keyword Search in Action

```
response = articles.query.bm25(  
    query="Vancouver hotel capacity",  
    limit=3  
)
```

BM25 keyword search

```
for o in response.objects:  
    print(o.properties)
```

Hybrid Search in Action

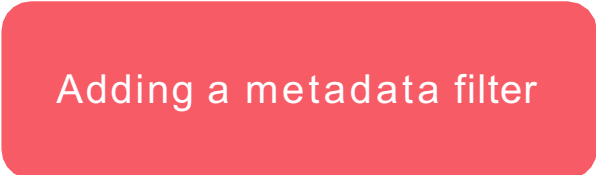
```
response = articles.query.hybrid(  
    query="Vancouver hotel capacity",  
    alpha=0.25,  
    limit=3,  
)  
  
for o in response.objects:  
    print(o.properties)
```

Hybrid search

Weighs the hybrid search
to 25% vector, 75% keyword

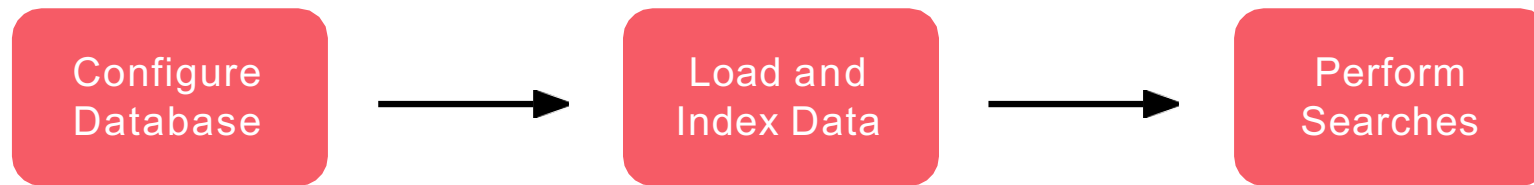
Filtered Search in Action

```
response = articles.query.hybrid(  
    query = 'History of urban development in Vancouver',  
    filters = Filter.by_property('title').contains_any(['Vancouver']),  
    alpha = 0.3,  
    limit = 4  
)  
  
for o in response.objects:  
    print(o.properties)
```



Adding a metadata filter

Complete Workflow



Using your own prompts

Your own system prompt

Use these principles to construct your own system prompt

- Instruct your LLM to respond in great detail OR answer questions succinctly
- For RAG applications, tell the language model to:

Use only retrieved documents to answer prompts
Judge whether a document is relevant
Cite sources in its response
- System prompts are added to every prompt your LLM processes

Prompt Templates

You're ready to build your augmented prompt

- Use a well-considered template since prompts include many pieces of information
- Templates provide structure and decide where content gets injected

```
# System Instructions
{system_prompt} # behavioral guidance

# Conversation History
# if conversation_history exists
User: {message_1}
Assistant: {response_1}
User: {message_2}
Assistant: {response_2}
...

# Retrieved Information
[DOCUMENT 1]
{chunk_1_text}
Source: {chunk_1_source}

...

# User Prompt

User: {user_query}
```

Prompt Template

System Instructions

You are an useful assistant for geographic information . Only use retrieved documents to answer.

- Use only retrieved documents
- Cite sources as [DOC X]
- Admit if information is missing

Conversation History

User: What is the largest city in Canada?

Assistant: Toronto is Canada's largest city with a population of 2.9 million [DOC 1]

Retrieved Documents

[DOC 1] Ottawa is the capital city of Canada, located in the province of Ontario [DOC 2] Ottawa became Canada's capital when Queen Victoria chose it in 1857

[DOC 3] The Parliament of Canada is located in Ottawa on Parliament Hill

Current Query

What is the capital of Canada?

Lecture References

- **Retrieval Augmented Generation (RAG) – Deeplearning.ai Course (Modules 1-4)**